

Tracker Build Map

Where every part lives, what it does, and what to touch to change it. The companion page argues *why* the design is this shape; this one is for when you already know and just need to find the file.

01 What actually runs today

Read this before planning work. Two of the five pieces are tested; the firmware is structured but unbuilt, because there is no ESP32 toolchain on the machine this was written on.

SUPABASE/MIGRATIONS Postgres schema + RLS + cron validated 59 statements, parsed with libpg_query. RLS on all 14 tables.	SUPABASE/FUNCTIONS ingest · dispatch · locate written, not deployed Deno/TypeScript. Device auth, escalation drainer, SMS downlink.	WEB/ Vercel dashboard written, not deployed Static. Supabase Auth + PostgREST + Realtime, no API layer.
SERVER/ FastAPI + SQLite reference tested end to end Still the best local dev target — SMS_PROVIDER=console exercises the whole ladder offline.	SURVEY/ANALYZE.PY Coverage report tested Verified against a synthetic route with a deliberately bad classroom.	SURVEY/SRC Phase 0 survey firmware written, uncompiled Complete logic — AT parsing, Wi-Fi scan, LittleFS log, waypoints.
TRACKER/SRC Worn device scaffold + stubs	SCANNER/SRC Gate attendance scaffold + stubs RC522 read loop works. Uplink and offline queue are TODO.	

Pins, tunables, feedback and motion are real. `modem.cpp` is the big TODO.

01B Two deployment paths — know which you are on

There are two working backends. They share the schema and the contracts and nothing else, so the first question before any change is *which one am I editing*.

	A — SUPABASE + VERCEL	B — FASTAPI
Database	Supabase Postgres	SQLite (Postgres-ready)
Device ingest	<code>functions/ingest</code> (Deno)	<code>app/main.py</code>
Escalation	<code>alert_queue</code> + <code>pg_cron/min</code>	asyncio task
Live updates	Supabase Realtime	WebSocket hub
Auth	Supabase Auth (MFA built in)	argon2 + TOTP, hand-rolled
Runs where	Nowhere — serverless	Fly / Railway / your laptop
Best for	Production	Local dev , and reading to understand the system

Why the ladder had to change. A Vercel function is killed the moment it returns its response, so a coroutine that sleeps for 300 s cannot exist. Pending rungs became rows with a `due_at`. That fixed a real latent weakness at the same time: the asyncio version lost every pending escalation on restart, and rows survive.

Rung 1 is deliberately not queued. `pg_cron` granularity is 60 s, so a queued rung due at `t+60` fires somewhere in `t+60..t+120`. Acceptable for the secondary SMS and the voice calls — but the first alert must never wait for a scheduler, so it is sent synchronously inside the ingest request.

PATH A — BEFORE ANY REAL DATA

RLS is the entire security model. Supabase exposes every table over PostgREST using the anon key, which ships in your frontend bundle. A table without RLS enabled is

readable by anyone who opens devtools. For a live database of a child's location that is not a misconfiguration — it is the worst outcome this project has.

`0002_rls.sql` enables RLS on all fourteen tables, with policies routed through `device_access`. Three tables — `alert_queue`, `access_log`, `integration_health` — get **no policies at all**: RLS on with nothing matching means they are invisible to everyone but the service role. Verify in the Table Editor before real data goes in.

The **service role key bypasses RLS entirely**. It lives in `supabase secrets` and Vercel environment variables. Never in `web/`, never in git, never in a client.

02 Contracts — change anything but these

Everything else in this system is a preference. These five are load-bearing: each one exists because breaking it produces a failure that looks like something else, which is the expensive kind.

1 Transmit the SOS at t+5 s with whatever fix exists

Indoors the GNSS branch never returns. Waiting for fix quality is a design that never fires inside a school. `SOS_TX_DEADLINE_MS`

2 HTTP 200 is the only ack

An event leaves the device's flash queue on 200 and on nothing else. "The request was sent" is not delivery, and a dropped SOS is unrecoverable.

3 Every event carries an id; ingest is idempotent

2G retries constantly. A redelivery must never become a second position on the map or a second attendance record. Enforced by `UNIQUE(event_id)`.

4 A position is never rendered without source, accuracy and age

A 12 m GNSS fix and a 2 km cell estimate are both "a pin" if you let them be. This is the most dangerous bug available in this system.

5 One press is one alert

The device texts the parent directly. When it reports `device_sms_sent`, the server skips its own t+0 SMS. Two texts for one press teaches parents to distrust the alert.

```

esp-tracker/
├── PLAN.md           the design and the reasoning – the source of truth
├── survey/           Phase 0. RUN THIS FIRST.
│   ├── src/main.cpp   coverage logger: cell + CSQ + Wi-Fi + GNSS, to flash
│   └── analyze.py      CSV → go/no-go report; --geolocate closes gate 2
├── tracker/          the worn device
│   ├── include/pins.h  pin map. ADC1-only and RTC-GPIO traps encoded here
│   ├── include/config.h every tunable. Start here for behaviour changes
│   └── src/
│       ├── sos.cpp      the five-second rule
│       ├── feedback.cpp LED + piezo cues, non-blocking, silent on SOS
│       ├── motion.cpp   cadence gate: cell tier + Wi-Fi Jaccard tier
│       ├── locator.cpp  fix fusion across the four sources
│       ├── modem.cpp    SIM800L: AT, HTTPS, SMS both ways ← biggest TODO
│       ├── gps.cpp      NEO-6M NMEA
│       └── store.cpp     flash queue, exponential backoff
├── scanner/          RFID attendance at the school gate
│   └── src/reader.cpp   RC522 UID read + debounce (working)
├──
├── supabase/          PATH A – production
│   ├── migrations/0001_schema.sql the data model. Read this first
│   ├── migrations/0002_rls.sql    ← the entire security model
│   ├── migrations/0003_cron.sql   dispatch every minute, prune nightly
│   └── functions/
│       ├── ingest/               device bearer auth; 200 IS the ack; rung 1 fires here
│       ├── dispatch/             drains alert_queue; ack cancels remaining rungs
│       └── locate/                parent-triggered SMS downlink, JWT + rate limited
├── web/                PATH A – Vercel. Static, no build step
│   ├── config.js         anon key (public – but only because RLS is on)
│   └── app.js             Leaflet, SOS banner, place picker, Realtime
├──
└── server/             PATH B – FastAPI reference. Best local dev target
    ├── schema.sql        SQLite twin of 0001_schema.sql
    ├── app/alerts.py      the asyncio ladder (what alert_queue replaced)
    └── app/main.py        all routes in one file

```

The common modifications, and the one thing to be careful about in each.

Change how often it reports

```
tracker/include/config.h
CADENCE_*_MS
```

Each report is a modem wake at up to 2 A. Halving the interval roughly doubles the battery cost of routine tracking.

Make the SOS button easier or harder to press

```
config.h → SOS_HOLD_MS
SOS_CANCEL_WINDOW_MS
```

Below ~1.5 s you start getting pocket false alarms, and enough of those teach parents to ignore the alert.

Change the escalation timing or channels

```
A: functions/ingest → alert_queue insert
A: functions/dispatch
B: server/app/config.py
```

Rung 1 fires inside ingest and must stay there — queued rungs inherit cron's ± 60 s. Voice costs money; keep it last.

Add or loosen a data access rule

```
supabase/migrations/0002_rls.sql
```

The highest-risk file in the repo. Every policy routes through `device_access`. A table with RLS off is world-readable via the anon key.

Change how long data is kept

```
0003_cron.sql → prune_old_data()
```

Liability, not housekeeping. Wi-Fi scans go first and fastest — an AP trail is more revealing than the coordinates beside it.

Swap the SMS provider

```
functions/_shared/sms.ts
supabase secrets set SMS_PROVIDER
```

Already abstracted. `console` prints instead of sending, so you can exercise the whole ladder for free.

Add a position source

```
tracker/src/locator.cpp
0001_schema.sql source CHECK
web/app.js SOURCE
```

Three places, all required. A source with no entry in `SOURCE` renders with the fallback colour and no label.

Tune indoor / outdoor detection

```
config.h MOTION_*
tracker/src/motion.cpp
```

Raising the Jaccard threshold makes it call "moving" more readily, which costs battery. `blind()` must keep its fixed fallback.

Change what the child feels or hears

```
tracker/src/feedback.cpp
config.h FEEDBACK_*
```

Patterns must be non-blocking — `sos::trigger()` is racing a 5 s deadline. And read `FEEDBACK_SILENT_SOS` before enabling audio on SOS cues.

Fit the vibration motor when it arrives

```
config.h FEEDBACK_USE_MOTOR
PIN_FEEDBACK (13)
```

Wiring change only — MOSFET + flyback diode, never straight off the pin. Set `FEEDBACK_ACK_HOLD_MS` back to 0 for a one-shot.

Add a dashboard panel

```
web/index.html
web/app.js
```

On path A you usually need no backend change at all — query Postgres directly and let RLS filter it. That is the point of the policies.

Deploy

```
supabase db push
supabase functions deploy
cd web && vercel --prod
```

`ingest` and `dispatch` need `--no-verify-jwt`: devices are not Auth users and cron calls with the service role.

Tighten escalation below 60 s

```
0003_cron.sql
functions/dispatch
```

Replace the cron job with delayed callbacks (QStash schedules to the second) and keep the same `alert_queue` table.

Add an SMS command from the dashboard

```
functions/_shared/sms.ts
functions/locate
tracker/src/modem.cpp
```

Authenticate it with `SMS_CMD_SECRET` and rate-limit it. The device number is a remote-control interface for anyone who learns it.

05 Pin maps

TRACKER SIGNAL	GPIO	NOTE
SIM800L TX → ESP32 RX	16	UART2
ESP32 TX → SIM800L RX	17	Level-shift this line — SIM800L RX is not 3.3 V tolerant
SIM800L PWRKEY	23	Pulse low ~1 s to toggle
SIM800L DTR	19	Held low to wake from <code>AT+CSCLK=1</code>
SIM800L power gate	32	High-side MOSFET
NEO-6M TX / RX	25 / 26	UART1, remapped off the flash pins
NEO-6M power gate	27	V_BCKP stays powered, or every fix is a 27 s cold start
SOS button	33	RTC-capable — required for deep-sleep wake. Active low
Feedback LED / motor	13	LED now, motor later — same pin
Piezo	12	Silent on SOS cues by default
Battery sense	34	ADC1 only — ADC2 cannot be read while Wi-Fi is active

SCANNER SIGNAL	GPIO	NOTE
RC522 SS / SCK / MOSI / MISO	5 / 18 / 23 / 19	SPI. 3.3 V only — 5 V destroys the module
RC522 RST	4	Moved off 22 so I ² C keeps the standard pair

SCANNER SIGNAL	GPIO	NOTE
DS3231 / OLED	21 / 22	I ² C. The RTC is not optional — see below
Buzzer / LED ok / LED err	25 / 26 / 27	The child must know the tap took

06 Tuning knobs

SETTING	DEFAULT	AFFECTS
<code>SOS_TX_DEADLINE_MS</code>	5000	Contract 1. Raising it means the SOS waits for a fix that, indoors, never comes
<code>SOS_HOLD_MS</code>	2000	False-alarm rate vs. how fast a frightened child can trigger it
<code>SOS_MAX_PER_HOUR</code>	5	Stuck-button protection. Suppressed events still raise an operator alert
<code>CADENCE_MOVING_MS</code>	120000	Battery, directly. Every report is a 2 A modem wake
<code>MOTION_SIMILAR_THRESHOLD</code>	0.55	Jaccard on the BSSID set. Higher = calls "moving" more often = more battery
<code>FEEDBACK_SILENT_SOS</code>	true	Safety. A beeping device announces a child who just called for help
<code>FEEDBACK_ACK_HOLD_MS</code>	20000	Set to 0 when the motor is fitted — haptics land instantly, LEDs must be looked at
<code>BATCH_MAX_EVENTS</code>	60	How far the 3–8 s TLS handshake is amortised
<code>ESCALATION</code> (<i>server</i>)	0/0/60/180/300 s	The alert ladder. Voice rungs cost money — keep them last
<code>STALE_AFTER_S</code> (<i>server</i>)	900	When a pin desaturates and stops claiming to be current
<code>RETAIN_SCANS_S</code> (<i>server</i>)	7 days	Wi-Fi scan history. An AP trail is more revealing than the coordinates
<code>dispatch-alerts</code> cron	* * * * *	Escalation granularity, and ~43K function calls/month — more than your ingest traffic

SETTING	DEFAULT	AFFECTS
<code>prune_old_data()</code>	7 / 90 / 30 days	Wi-Fi scans / locations / health. SOS events are kept forever, on purpose
<code>RATE_LIMIT_S</code> (<i>locate</i>)	60	Stops repeated "locate now" taps flattening the battery
<code>COOKIE_SECURE</code> (<i>path B only</i>)	1	Set 0 for local HTTP dev only . Never on anything reachable

07 What is provisional

Four parts are stand-ins for something else. Each is a deliberate trade, and each is worth revisiting when stock allows.

WANTED	FITTED	WHAT IT COST
SIM7080G (LTE-M)	SIM800L (2G)	No modem TLS, battery 5 days → 2–3. Bought SMS as an unkillable channel. Correct part for PH regardless
LIS3DH accelerometer	Wi-Fi + cell motion gate	~0.5% battery and no interrupt wake. Arguably a <i>better</i> signal — jostling isn't movement
Vibration motor	LED + piezo	The one substitution that is purely worse. Haptics are the only channel both silent and felt
Google FMDN fallback	nothing	A tracker that is off or out of coverage is simply gone. Deliberate — see the companion page

BEFORE TOUCHING THE FEEDBACK CODE

SOS cues are silent on purpose. A piezo is a better stand-in than an LED for most things, because sound reaches a child who is not looking. But an SOS is the one case where being heard can be the danger: a child presses the button precisely when someone is frightening them, and a device that then beeps announces both the child and the fact that they called for help.

So the piezo carries boot, low battery, drill mode and "locate now"; the SOS cues stay visual. If you set `FEEDBACK_SILENT_SOS = false`, understand that is the trade you are making.

08 Next, in order

#	DO	UNTIL
1	Wire the survey rig, walk the school run on Globe, then Smart	Registered $\geq 98\%$, CSQ median ≥ 15
2	<code>analyze.py --geolocate</code> on the same log	Provider resolves ≤ 50 m inside the school
3	Fill in <code>tracker/src/modem.cpp</code>	AT attach, HTTPS POST, SMS both ways, <code>AT+CCLK?</code>
4	Run the migrations, verify RLS is on every table , deploy the functions	Table Editor shows "RLS enabled" 14x
5	SOS end to end, 50 real presses	$p95 \leq 20$ s; 20 indoor presses give a usable fix

Steps 1 and 2 are a weekend with parts you already have, and they decide whether the rest of the project is buildable. Everything after them assumes the answer was yes.

Companion to PLAN.md and the architecture page · v4. Migrations validated with `libpg_query`; the FastAPI reference and `analyze.py` are tested. Firmware and Edge Functions are written but not yet run – expect to fix a header, an import or a library version on first build. Firmware here is uncompiled – expect to fix a header or a library-version mismatch on the first `pio run`. The server and `analyze.py` are tested. Do not expose the server until `/ws/live` authenticates its socket.